

Amirali Abazari - Valentin Gaucher - Jake Strube
Arizona State University, Ira.A.Fulton Schools of Engineering
RAS 557: Foldable Robotics

Final Project - Bio Inspired Jumping Flea Robot

Introduction

In small scale biological systems, muscle power is often insufficient to generate the high accelerations required for ballistic movements. To overcome this physiological limit, organisms such as fleas (Siphonaptera) employ a power amplification strategy known as Latch-Mediated Spring Actuation (LaMSA). This mechanism decouples the relatively slow phase of energy production from the rapid phase of power release. Muscles contract slowly to store potential energy in elastic elements specifically, a resilin pad located in the thorax, while a mechanical latch maintains the system in a high energy state. Once unlatched, the stored energy is released on the timescale of milliseconds, bypassing the velocity limits of direct muscle actuation. In the flea, this energy is routed through a complex linkage system acting as a lever, transmitting forces through the tibia and tarsus to the ground. This transmission results in takeoff velocities ranging from 1.3 to 1.9 m/s and controlled body rotation, governed largely by the passive geometry of the legs rather than active control during flight.

Translating these biological principles into miniature robotics presents unique challenges. Traditional rigid body robots often rely on heavy motors and high bandwidth feedback controllers to manage dynamics. However, at the gram and millimeter scales, the "mechanics-first" design philosophy dominates. Recent advancements in foldable and origami-inspired robotics demonstrate that performance metrics, such as jump length, stability, and landing accuracy, can be encoded directly into the robot's physical structure. By manipulating the geometry of folds and the composition of laminate layers, designers can program specific stiffness profiles and impedance properties into the chassis itself.

This study aims to develop a bio-inspired, foldable jumping robot that mimics the kinematic topology and energy release dynamics of the flea. The approach integrates two complementary methodologies:

- **Computational Optimization:** A planar rigid body simulation was developed using the MuJoCo physics engine to model the contact dynamics and optimize the linkage geometry (specifically the spring attachment points) for maximum forward range.
- **Robot Design:** A physical prototype was built using a five-layer laminate manufacturing process. This design incorporates bistable metal tape as a tunable stiffness element and a mechanism to replicate the biological latching function.

By bridging the gap between theoretical optimization and physical implementation, this project seeks to validate how passive structural compliance and geometric tuning can replace complex active control in achieving robust, high-performance jumping at the miniature scale.

The Project Video can be find here: <https://youtu.be/GWAMmjKtZHU>

Background Research

How animals achieve jump power and damping

Animals reach extreme lift-off power by *decoupling* muscle work from push-off using *latch-mediated spring actuation* (LaMSA): muscles load elastic elements slowly, a latch holds position, and rapid unlatching converts stored elastic energy into kinetic energy over a short interval, yielding an effective power that can exceed muscle power limits [1, 2, 3]. In fleas, Sutton & Burrows resolved the transmission path: energy is stored in a resilin spring in the thorax and routed through a lever (see Figure 1). The push-off occurs via the tibia/tarsus, with release on the order of ~ 1 ms producing ~ 1.3 – 1.9 m/s take-off velocities and controlled body rotation set by leg geometry and contact timing [4]. Miniature, flea-inspired catapults formalize the same architecture (elastic storage + latch + transmission): tuning the *lever arms* and *latch geometry* sets take-off speed and angle at millimeter scales, and published prototypes demonstrate body-length-normalized heights comparable to biological benchmarks [5, 6, 7].

Real tissues are *viscoelastic*, so elastic elements (resilin/cuticle) dissipate a fraction of stored energy quantified by the loss factor. This intrinsic hysteresis behaves as passive damping that shapes rebound and landing dynamics [8, 9, 10]. Quantitative analyses of ultrafast LaMSA events show that *nonlinear spring curves* and *internal damping* strongly influence the achievable kinematics and contact forces [11].

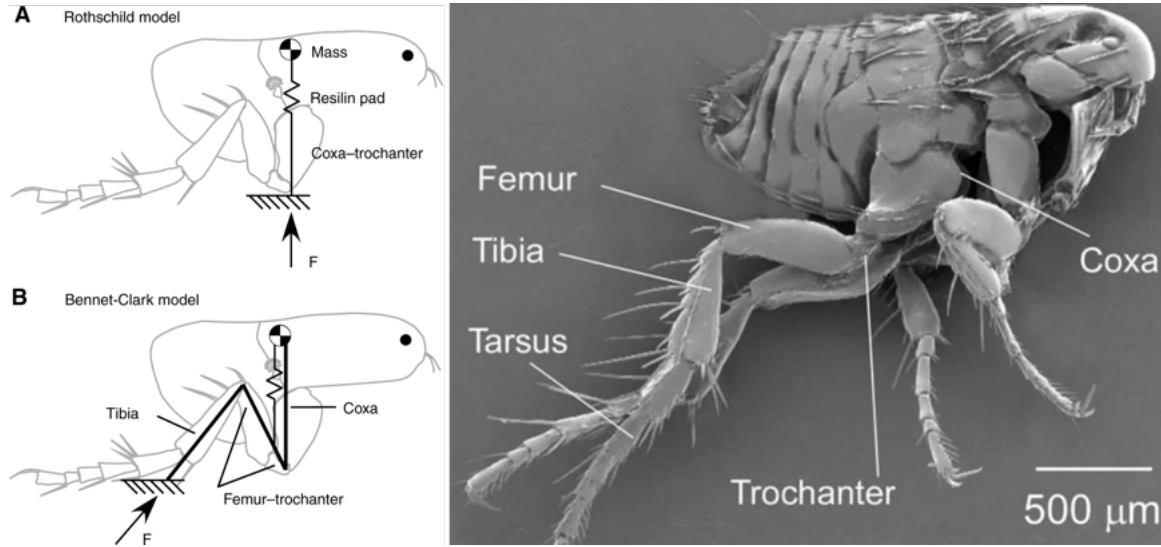


Figure 1: Flea LaMSA anatomy and kinematics: slow elastic loading, latch hold, rapid unlatch. The force transmitted to ground by tibia/tarsus [4].

Review on foldable jumping robots

Across origami and laminate robots, the dominant strategy is to *encode structural impedance* meaning stiffness, energy storage, and even damping directly into the sheet architecture so that jump output and landing dynamics follow from passive mechanics rather than heavy controllers [12, 13, 14].

We first identify geometry programmed springs. REBOund uses a REBO tessellation as the body spring. A pseudo rigid body modeling (PRBM) approach links fold geometry to force/displacement and stored energy, enabling small geometric changes to tune jump height. Prototypes reached ≈ 97 123 mm while the folded sheet serves as the primary elastic engine [12]. TMP bellows exploit *strain softening* to store more energy than a linear spring and improve airtime and height ($\approx +9\%$ airtime, $\approx +13\%$ height), provided fold line hysteresis is mitigated (via improved creasing) [15]. Curved crease origami can switch *in situ* among positive, near zero, and negative stiffness by activating different crease sets, offering intrinsic variable stiffness (and thus passive damping shaping) in thin shells [16].

We have robots structures with variable stiffness/damping add ons. Laminar (layer) jamming laminates provide large, controllable increases in both stiffness and damping under vacuum. Analytical and FE models make the achievable ranges and transition speeds designable, making jamming a natural structural clutch/damper to pair with elastic origami springs [14, 17].

At the mechanism scale, direct encoding of crease stiffness in an origami inspired jumping leg lets designers redistribute laminate materials to set the ratio between short and long hinges, sculpt the force/displacement curve, and improve jump output without adding separate mechanisms [13].

At gram scales, Moobot demonstrates that careful structural transmission plus minimal sensing yields omnidirectional jumps with < 1.5 cm landing error at ~ 6 g and ~ 5 cm height, underscoring that mechanical encoding can carry much of the burden for trajectory accuracy [18]. For larger, high agility systems, stance phase balance and angular momentum management (e.g., Salto 1P) reduce range error by ~ 1.6 5.8 $\times$ compared to flight phase control alone useful benchmarks for what passive structure must withstand when feedback is deliberately kept light [19].

Finally, flea inspired catapults at millimeter scale consistently show that latch design and lever ratios dominate attainable takeoff velocities, reinforcing a mechanics first design stance for foldable jumpers [5, 6, 7]. We show some of those concepts Figure 2

Key Data and Design Criteria

For a *foldable* jumper with *intrinsic* stiffness/damping control (structure-level, not controller-level), the literature converges on four mechanics-first levers: (i) use *nonlinear* springs to boost stored energy and sculpt release [15], (ii) enable *in-situ* variable stiffness (including near-zero/negative regions) to tailor push-off impulse and soften landing [16], (iii) add *structural* damping/clutching to quench ring-down and chatter [14, 17], and (iv) rely on *geometric* tuning so jump height/angle follow directly from fold parameters with minimal actuation [12, 13]. We anchor numeric targets

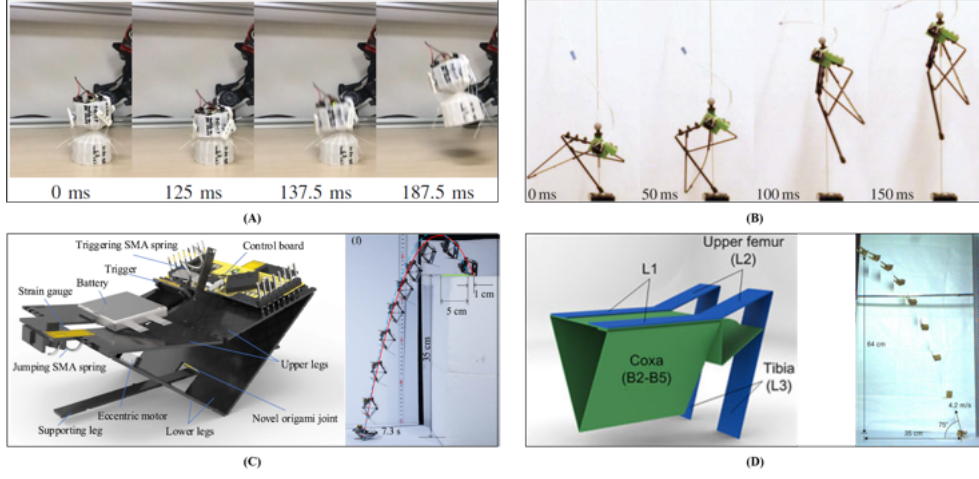


Figure 2: **Foldable jumping designs at a glance.** (A) REBOund body spring with geometry programmable jump height [12]. (B) Laminate origami leg with directly encoded crease stiffness/non-linearity [13]. (C) Moobot: accurate miniature origami jumper [18]. (D) Flea inspired catapults illustrating lever+latch dominance and torque reversal variants [5, 6, 7].

where available and convert them into crisp design checks below (Table 1).

From there, we propose a quantitative baselines used to size targets. Let m be robot mass, E the elastic energy released at push-off, and Δt the push-off window. Then

$$v_0 = \sqrt{\frac{2E}{m}}, \quad h = \frac{E}{mg}, \quad (1a)$$

$$J = \int_0^{\Delta t} F_z(t) dt \approx mv_0, \quad \bar{F} = \frac{J}{\Delta t}. \quad (1b)$$

Flea biomechanics demonstrate millisecond-scale release and $v_0 \sim 1.3\text{--}1.9\text{ m/s}$ at mg scale [4]. Foldable robots at gram scale report 5–12 cm jumps with geometry-dominant mechanics [18, 12]. We therefore size criteria so that (a) geometry and spring curve dominate E and v_0 , (b) stiffness/damping are *actively* switchable across stance/landing and (c) latch timing is fast enough that Δt does not bottleneck impulse.

Table 1: Evidence-based design choices for a structure-first foldable jumper. Targets are stated so they can be checked experimentally.

Criterion	Target / Range	Evidence and rationale
Nonlinear elastic engine	Use a strain-softening spring. Aim for lab jumps of 0.10–0.12 m while keeping fold hysteresis low.	TMP bellows with designed softening increased air-time and height relative to quasi-linear baselines, showing that shaping the force–displacement curve raises usable stored energy [15]. REBOund reached 97–123 mm with a geometry-only spring, confirming feasibility in a foldable format [12].
In-situ stiffness switching	Provide positive, near-zero, and negative effective stiffness states. Achieve at least a fivefold stiffness span. Switch within 20 ms.	Curved-crease origami allows switching among stiffness regimes within a single sheet, enabling a stiff push-off and a compliant landing without adding mechanisms [16]. The requested span and timing separate stance and landing behavior in practice.
Structural damping / clutch (layer jamming)	Tunable dissipation with low damping during push-off and higher damping at landing (e.g., $\zeta \approx 0.05$ – 0.15 vs. 0.4 – 0.7). Seek at least an order-of-magnitude stiffness increase when engaged.	Laminar jamming increases both stiffness and damping under vacuum. Analytical and FE models map laminate design to achievable ranges and transition speeds, making it a practical clutch/damper paired with an elastic spring [14, 17].
Geometry-to-height mapping	Monotonic relation between a fold parameter and jump height. Resolve at least three distinct height setpoints repeatably.	REBOund’s pseudo-rigid-body model links fold geometry to stored energy and jump outcomes. Untethered demonstrations achieved 97–123 mm by geometry alone, indicating that height can be scheduled through design rather than control [12].
Stiffness encoding in leg/crease	Short-to-long hinge stiffness ratio of at least 3:1, with a measurable increase in take-off speed over a uniform-hinge baseline.	Directly encoding crease stiffness in an origami-inspired jumping leg reshapes the force–displacement curve and improves take-off performance without additional mechanisms [13].
Latch and lever design (mm–cm scale)	Unlatch within 20 ms. Choose lever ratios to set the desired take-off angle and avoid premature slip.	Flea-inspired micro-catapults show that latch dynamics and lever tuning dominate attainable take-off velocity at small scales [5, 6, 7]. Flea biomechanics provide a biological reference with millisecond-scale release [4].
Scale and accuracy benchmark	Mass 6–10 g. Jumps of 5–12 cm. Landing error below 1.5 cm in benchtop tests.	Moobot (origami + SMA) achieved sub-centimeter-scale accuracy at 6 g and roughly 5 cm jumps, indicating that a mechanics-dominant transmission with minimal sensing can deliver precise landings [18].
Stance-phase accuracy (context)	Prefer stance-phase balance for precision launches. Expect a 1.6 – $5.8\times$ reduction in range error relative to flight-phase control alone.	Salto-1P demonstrated that stabilizing during stance sharpens range accuracy, setting expectations for what the passive structure must accommodate when feedback is kept light [19].

Preliminary Concept

Based on our literature review and the Foldable Robotics Course we propose the following geometries for the flea mechanism

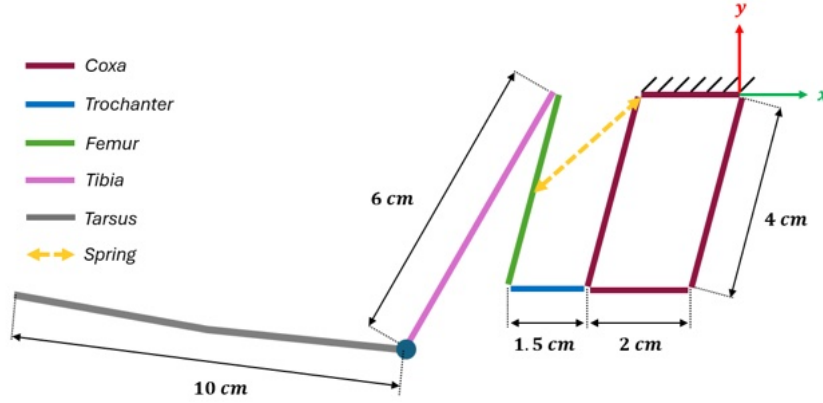


Figure 3: Geometric Identification for the flea bio-inspiration

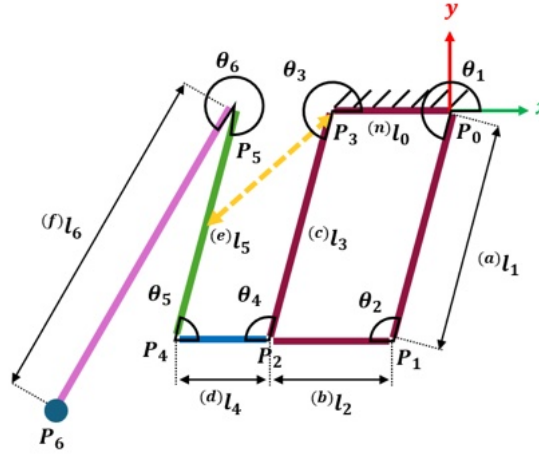


Figure 4: Geometry simulated for the flea robot

We choose to model the Tarsus as a single point with high friction. This choice was motivated by [20] where the Tarsus is an effective surface with "gripping hair" that provide attachment and therefore high friction.

Simulations and Design Optimization

MuJoCo Modeling and Kinematics

To investigate the jump capabilities of the flea-inspired mechanism, a planar rigid-body simulation was developed using the MuJoCo physics engine [21]. The robot's kinematic chain is constrained to the $(X - Z)$ plane, reflecting the biological jumping motion of Siphonaptera. The model topology consists of a floating base ("Trunk") connected to a multi-link leg structure comprising a Coxa, Trochanter, Femur, and Tibia-Tarsus [20].

A critical feature of the design is the Coxa mechanism, modeled as a four-bar parallelogram linkage with bar lengths $L_1 = L_3 = 0.04$ m and $L_2 = L_0 = 0.02$ m. This geometric constraint ensures that the Trochanter maintains a fixed orientation relative to the Trunk during flexion, decoupling the leg's extension angle from the body's pitch. The Femur ($L_5 = 0.04$ m) and Tibia ($L_6 = 0.05$ m) are connected via revolute joints with damped limits to prevent hyperextension during the jump flight phase.

Energy storage is modeled via a massless linear actuator acting as a spring spanning the Coxa and

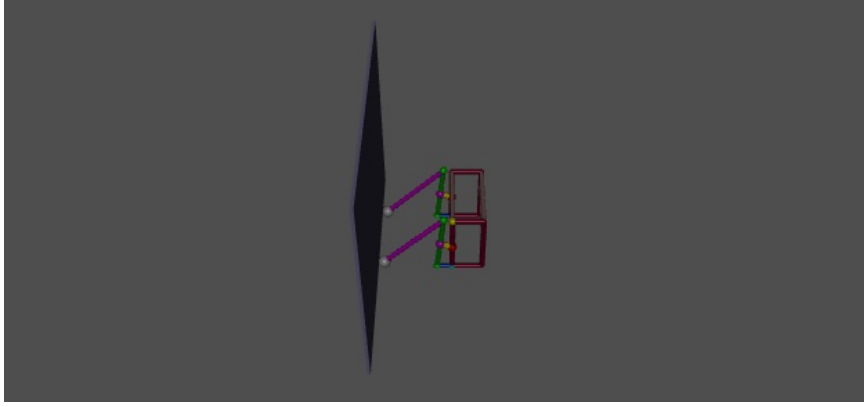


Figure 5: MuJoCo Model

Femur. Unlike biological muscle which actively contracts, the simulation approximates the rapid release of stored elastic energy by instantaneously modifying the tendon's rest length (L_0) at the onset of the simulation ($t = 0$). The potential energy U injected into the system is defined by:

$$U = \frac{1}{2}k(L_{\text{contract}} - L_{\text{extended}}) \quad (2)$$

where k is the spring stiffness and L_{extended} is the modified rest length inclusive of the pre-load δ . The contact dynamics are governed by a soft-contact model with a high friction coefficient ($\mu = 0.99$) at the Tarsus tip to maximize traction during the propulsive stroke. This high friction mimics the flea anatomy, where the flea "hairs" grip to the contact surface providing an attachment [20].

We provide a visual of the MuJoCO system Figure 5.

Baseline Performance and Trajectory Analysis

The simulation integrates the equations of motion over a duration of 2.0 seconds with a fixed timestep of 10^{-4} seconds. The jumping maneuver shows three distinct phases: the propulsive stroke, the ballistic flight, and the landing impact.

During the propulsive phase (approx. $t < 0.1$ s), the stored potential energy is rapidly converted into kinetic energy. The kinematic analysis reveals that the parallelogram linkage of the Coxa successfully directs the ground reaction forces (GRF) through the center of mass, minimizing backward pitch which is a common failure mode in monopodal jumpers.

Preliminary results using the initial geometric guesses yielded a stable parabolic trajectory. The Trunk segment traced a ballistic arc, reaching a peak before descending. The primary performance metric established for this study is the net forward displacement of the Trunk, denoted as x_{max} , measured at the point of first ground contact or system rest. This metric was used assuming the flea wants to move forward therefore the jump length is what matter.

We provide the trajectory of the simulation in MuJoCo Figure 6, 7.

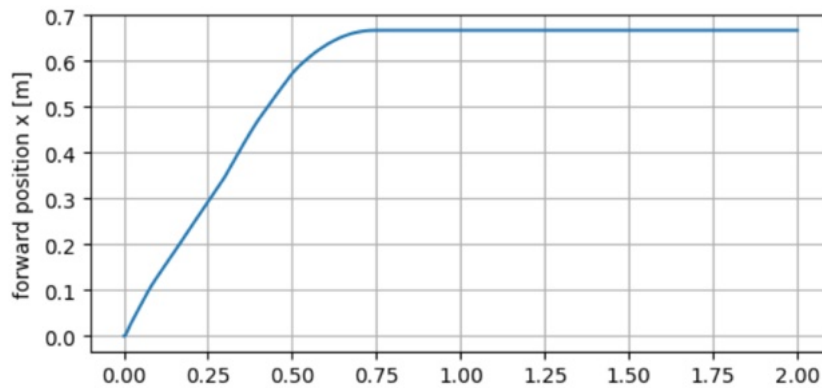


Figure 6: MuJoCO simulation for the Forward Motion

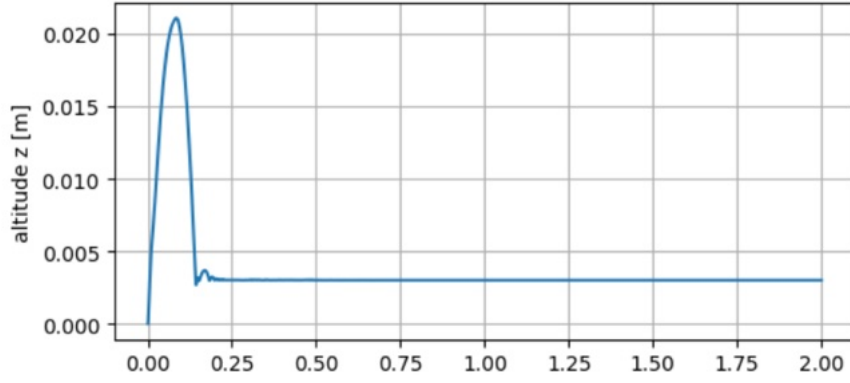


Figure 7: MuJoCO simulation for the Altitude Motion

Design Optimization

To enhance the robot's range, an optimization study was conducted focusing on the geometric configuration of the energy storage element. The transmission of the spring spring energy during extension is a function of the spring's attachment points. We defined two dimensionless design variables, α and β , representing the normalized attachment positions along the Coxa and Femur segments, respectively.

Methodology

We implemented a nested grid search algorithm to sweep the parameter space $\alpha, \beta \in [0.1, 1.0]$. The optimization loop regenerated the XML model structure for each permutation, ran the forward dynamics simulation, and extract the terminal x_{max} . This was followed by a refined local search centered on the global maximum to minimize discretization error. For this analysis, we guess an initial stiffness $k = 150$ M.m amd run the analysis.

Simultaneously, a sweep of the stiffness parameter k was performed on a logarithmic scale up to 150 N/m to verify that the selected spring constant did not induce instability or numerical divergence.

Optimization Results

The response to the sweep revealed a clear convexity, indicating a single global optimum rather than multiple local maxima. The refined optimization identified the ideal configuration at:

$$\alpha^* = 0.44 \quad \beta^* = 0.50$$

We provide the heatmap of the optimization Figure 8

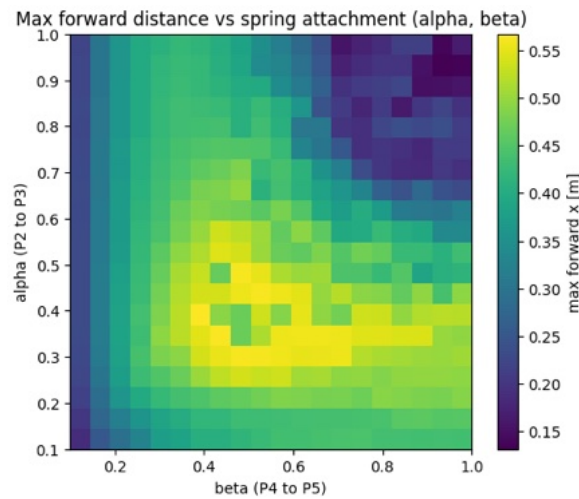


Figure 8: Heatmap of the optimization process

Under these parameters, the model achieved a predicted maximum forward distance of 0.5729 m. This configuration places the femoral attachment point exactly at the segment's midpoint, balancing the moment arm length against the angular velocity requirements of the joint. We provide the stiffness optimization Figure 9

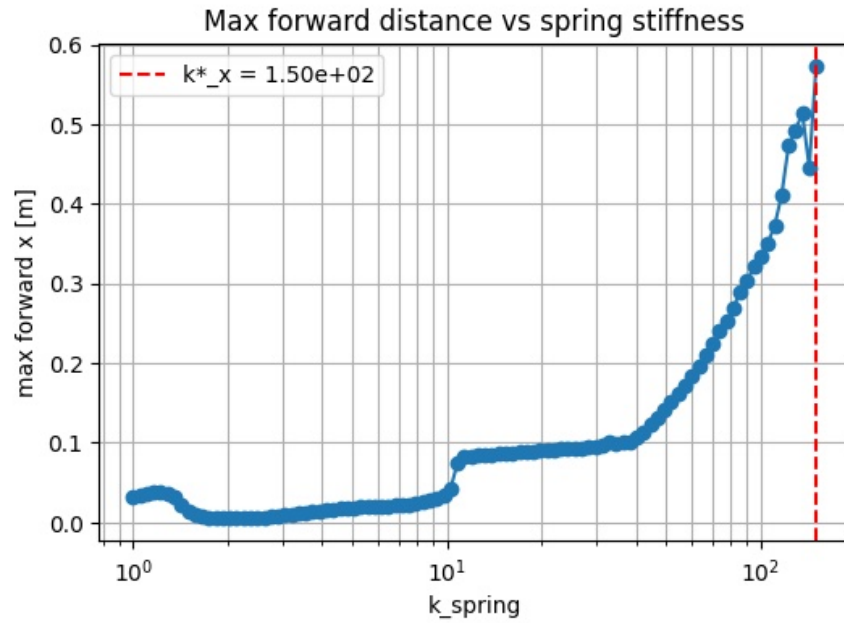


Figure 9: Sweep of spring stiffness for optimized spring placment

MuJoCo Jupyter Notebook Code

We provide our code here:

MuJoCo Simulations

```
In [20]: import mujoco
import mediapy as media
import numpy as np
import math
import matplotlib.pyplot as plt
```

Defining Geometry

```
In [21]: # --- Coxa ---
L1 = 0.04      # (a) l1 front bar of Coxa
L2 = 0.02      # (b) l2 bottom bar
L0 = L2        # (n) l0 top bar (parallelogram -> same as l2)
L3 = L1        # (c) l3 rear bar (parallelogram -> same as l1)

# --- Legs ---
L4 = 0.01      # (d) l4 Trochanter
L5 = 0.04      # (e) l5 Femur
L6 = 0.05      # (f) l6 Tibia-tarsus (P5-P6)

# --- Spring ---
k_spring = 150  # N/m spring between P3 and mid femur
d_spring = 0.5  # N·s/m damping
alpha = 0.44    # Spring placement on Coxa
beta = 0.50     # Spring placement on Femur

delta = 0.07    # extra rest length (so current length < rest length)
```

Defining the XML file

```
In [22]: def make_xml(alpha, beta, k_spring):
    return f"""
<mujoco model="flea_leg_forward">

  <compiler angle="degree" coordinate="local"/>

  <!-- 2D in x-z; gravity in -z -->
  <option gravity="0 0 -9.81" timestep="1e-4">
    <flag contact="enable"/>
  </option>

  <!-- Default: low friction everywhere; we'll override floor-foot -->
  <default>
    <geom contype="1" conaffinity="1" condim="3"
      friction=".1 .3 .3"
      solimp=".999 .999 .001"
      solref=".002 1"
      margin="0.001" size="0.002"/>
  </default>

  <worldbody>

    <!-- Ground plane z=0; collides with feet -->
    <body name="floor">
      <geom name="floor_geom" type="plane" size="1.5 0.5 0.002"
        rgba="0.8 0.8 0.8 1"
        contype="1" conaffinity="1" condim="3"/>
    </body>

    <!-- Vertical block near P6 to push off -->
    <body name="wall" pos="-0.07 0 0">
      <!-- thin vertical box in x-z plane -->
      <geom name="wall_geom" type="box"
        pos="0 0 0.03"
        size="0.001 0.09 0.1"
        rgba="0.7 0.7 0.9 1"
        contype="1" conaffinity="1" condim="3"/>
    </body>

    <!-- Trunk: can move in x and z (forward jump) -->
    <body name="trunk" pos="0 0 0">
      <!-- planar DOFs -->
      <joint name="x_trunk" type="slide" axis="1 0 0"/>
      <joint name="z_trunk" type="slide" axis="0 0 1"/>

      <!-- small invisible mass at trunk origin -->
      <geom name="trunk_mass" type="sphere"
        pos="0 0 0"
        size="0.002" rgba="0 0 0 0"
        mass="0.02"/>

      <!-- Coxa root P0 at height L1 above trunk origin -->
```

```

<body name="coxa_root" pos="0 0 {L1:.4f}">
  <!-- 01: overall coxa orientation (shared by both rails) -->
  <joint name="theta1" type="hinge" axis="0 1 0"/>

  <!-- Coxa parallelogram P0-P1-P2-P3, with duplicated rail at y=0.05 -->
  <body name="coxa" pos="0 0 0">

    <!-- FRONT BAR (P0 -> P1): right rail (y=0) -->
    <geom name="coxa_l1" type="capsule"
      fromto="0 0 0 0 0 -{L1:.4f}"
      size="0.0018" rgba="0.6 0.0 0.2 1"/>
    <!-- copy rail at y=+0.05 -->
    <geom name="coxa_l1_copy" type="capsule"
      fromto="0 0.05 0 0 0.05 -{L1:.4f}"
      size="0.0018" rgba="0.6 0.0 0.2 1"/>
    <!-- cross-bar P0-P0' -->
    <geom name="coxa_bar_P0" type="capsule"
      fromto="0 0 0 0 0.05 0"
      size="0.0014" rgba="0.8 0.6 0.6 1"/>
    <!-- cross-bar P1-P1' -->
    <geom name="coxa_bar_P1" type="capsule"
      fromto="0 0 -{0.3*L1:.4f} 0 0.05 -{0.3*L1:.4f}"
      size="0.0014" rgba="0.8 0.6 0.6 1"/>

    <!-- BOTTOM BAR (P1 -> P2): right rail -->
    <geom name="coxa_l2" type="capsule"
      fromto="0 0 -{L1:.4f} -{L2:.4f} 0 -{L1:.4f}"
      size="0.0018" rgba="0.6 0.0 0.2 1"/>
    <!-- copy rail at y=+0.05 -->
    <geom name="coxa_l2_copy" type="capsule"
      fromto="0 0.05 -{L1:.4f} -{L2:.4f} 0.05 -{L1:.4f}"
      size="0.0018" rgba="0.6 0.0 0.2 1"/>
    <!-- cross-bar P2-P2' -->
    <geom name="coxa_bar_P2" type="capsule"
      fromto="-{L2:.4f} 0 -{0.3*L1:.4f} -{L2:.4f} 0.05 -{0.3*L1:.4f}"
      size="0.0014" rgba="0.8 0.6 0.6 1"/>

    <!-- REAR BAR (P2 -> P3): right rail -->
    <geom name="coxa_l3" type="capsule"
      fromto="-{L2:.4f} 0 -{L1:.4f} -{L2:.4f} 0 0"
      size="0.0018" rgba="0.6 0.0 0.2 1"/>
    <!-- copy rail at y=+0.05 -->
    <geom name="coxa_l3_copy" type="capsule"
      fromto="-{L2:.4f} 0.05 -{L1:.4f} -{L2:.4f} 0.05 0"
      size="0.0018" rgba="0.6 0.0 0.2 1"/>
    <!-- cross-bar P3-P3' -->
    <geom name="coxa_bar_P3" type="capsule"
      fromto="-{L2:.4f} 0 0 -{L2:.4f} 0.05 0"
      size="0.0014" rgba="0.8 0.6 0.6 1"/>

    <!-- TOP BAR (P3 -> P0): right rail -->
    <geom name="coxa_l0" type="capsule"
      fromto="-{L2:.4f} 0 0 0 0 0"
      size="0.0018" rgba="0.6 0.0 0.2 1"/>
    <!-- copy rail at y=+0.05 -->
    <geom name="coxa_l0_copy" type="capsule"
      fromto="-{L2:.4f} 0.05 0 0 0.05 0"
      size="0.0018" rgba="0.6 0.0 0.2 1"/>

    <!-- Reference sites P2 and P3 (on right rail y=0) -->
    <site name="P2_site" pos="-{L2:.4f} 0 -{L1:.4f}"
      size="0.0025" rgba="0 1 1 1"/>
    <site name="P3_site" pos="-{L2:.4f} 0 0"
      size="0.0025" rgba="1 1 0 1"/>

    <!-- alpha interpolation on coxa edge P2->P3, right rail (y=0) -->
    <site name="spring_coxa_site"
      pos="-{L2:.4f} 0 {(-L1 + alpha*L1):.4f}"
      size="0.0030" rgba="1 0 0 1"/>

    <!-- alpha interpolation for copy rail at y=+0.05 -->
    <site name="spring_coxa_site_copy"
      pos="-{L2:.4f} 0.05 {(-L1 + alpha*L1):.4f}"
      size="0.0030" rgba="1 0 0 1"/>

    <!-- Trochanter from P2 (rigid extension, NO joint) -->
    <body name="trochanter" pos="-{L2:.4f} 0 -{L1:.4f}">
      <!-- trochanter is a rigid extension of the coxa bottom -->
      <!-- right rail -->
      <geom name="troch_geom" type="capsule"
        fromto="0 0 0 -{L4:.4f} 0 0"
        size="0.0016" rgba="0.0 0.2 0.8 1"/>
      <!-- copy rail at y=+0.05 -->
      <geom name="troch_geom_copy" type="capsule"
        fromto="0 0.05 0 -{L4:.4f} 0.05 0"
        size="0.0016" rgba="0.0 0.2 0.8 1"/>

```

```

<!-- Femur from P4 (hinge joint here, shared) -->
<body name="femur" pos="{L4:.4f} 0 0">
  <joint name="theta5" type="hinge" axis="0 1 0"
    limited="true" range="-90 40"/>

  <!-- Femur: right rail -->
  <geom name="femur_geom" type="capsule"
    fromto="0 0 0 -0.01 0 {L5:.4f}"
    size="0.0016" rgba="0.0 0.8 0.0 1"
    mass="0.001"/>
  <!-- Femur: copy rail at y=+0.05 -->
  <geom name="femur_geom_copy" type="capsule"
    fromto="0 0.05 0 -0.01 0.05 {L5:.4f}"
    size="0.0016" rgba="0.0 0.8 0.0 1"
    mass="0.001"/>

  <!-- Reference sites for P4, P5 (right rail) -->
  <site name="P4_site" pos="0 0 0"
    size="0.0025" rgba="0 1 0 1"/>
  <site name="P5_site" pos="-0.01 0 {L5:.4f}"
    size="0.0025" rgba="0 1 0 1"/>

  <!-- Reference sites for copy rail (P4', P5') at y=+0.05 -->
  <site name="P4_site_copy" pos="0 0.05 0"
    size="0.0025" rgba="0 1 0 1"/>
  <site name="P5_site_copy" pos="-0.01 0.05 {L5:.4f}"
    size="0.0025" rgba="0 1 0 1"/>

  <!-- beta interpolation on femur P4->P5, right rail -->
  <site name="spring_femur_site"
    pos="{(-0.01*beta):.4f} 0 {(L5*beta):.4f}"
    size="0.0030" rgba="1 0 1 1"/>

  <!-- beta interpolation on femur P4'>P5', copy rail -->
  <site name="spring_femur_site_copy"
    pos="{(-0.01*beta):.4f} 0.05 {(L5*beta):.4f}"
    size="0.0030" rgba="1 0 1 1"/>

  <!-- Tibia/Tarsus from P5 -->
  <body name="tibia" pos="-0.01 0 {L5:.4f}">
    <joint name="theta6" type="hinge" axis="0 1 0"
      limited="true" range="-50 120.0"/>

    <!-- Tibia: right rail -->
    <geom name="tibia_geom" type="capsule"
      fromto="0 0 0 -0.02 0 -{L6:.4f}"
      size="0.0016" rgba="1 0.0 1 1"
      mass="0.005"/>
    <!-- Tibia: copy rail at y=+0.05 -->
    <geom name="tibia_geom_copy" type="capsule"
      fromto="0 0.05 0 -0.02 0.05 -{L6:.4f}"
      size="0.0016" rgba="1 0.0 1 1"
      mass="0.005"/>

    <!-- Foot at P6: right leg -->
    <geom name="foot" type="sphere"
      pos="-0.02 0 -{L6:.4f}"
      size="0.004" rgba="0.9 0.9 0.9 1"
      mass="0.001"
      contype="1" conaffinity="1"/>

    <!-- Foot copy at P6': second leg (y=+0.05) -->
    <geom name="foot_copy" type="sphere"
      pos="-0.02 0.05 -{L6:.4f}"
      size="0.004" rgba="0.9 0.9 0.9 1"
      mass="0.001"
      contype="1" conaffinity="1"/>
  </body>
</body>
</body>
</body>
</body>
</worldbody>

<!-- CONTACT BLOCK -->
<contact>
  <!-- High friction between floor and feet (P6, P6') -->
  <pair geom1="floor_geom" geom2="foot"
    friction=".99 .99 .99"/>
  <pair geom1="floor_geom" geom2="foot_copy"
    friction=".99 .99 .99"/>
  <!-- High friction between wall and foot (P6, P6') -->
  <pair geom1="wall_geom" geom2="foot"
    friction=".99 .99 .99"/>
  <pair geom1="wall_geom" geom2="foot_copy"
    friction=".99 .99 .99"/>

```

```

        friction=".99 .99 .99"/>
</contact>

<!-- Springs between interpolated Coxa and Femur sites on each rail -->
<tendon>
  <!-- Spring on original leg (y=0) -->
  <spatial name="Palpha_Pbeta_spring"
    stiffness="{k_spring}" damping="{d_spring}"
    width="0.002" rgba="1 1 0 1">
    <site site="spring_coxa_site"/>
    <site site="spring_femur_site"/>
  </spatial>

  <!-- Spring on copy leg (y=+0.05) -->
  <spatial name="Palpha_Pbeta_spring_copy"
    stiffness="{k_spring}" damping="{d_spring}"
    width="0.002" rgba="1 1 0 1">
    <site site="spring_coxa_site_copy"/>
    <site site="spring_femur_site_copy"/>
  </spatial>
</tendon>

</mujoco>
"""

```

Build model and data

```

In [23]: XML = make_xml(alpha, beta, k_spring)
model = mujoco.MjModel.from_xml_string(XML)
data = mujoco.MjData(model)
# IDs
trunk_bid = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_BODY, "trunk")
spring_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_TENDON, "Palpha_Pbeta_spring")

```

Set the crouched position

```

In [24]: # trunk position (planar)
x_trunk0 = 0.0 # forward position
z_trunk0 = 0.0 # trunk origin at ground

# desired angles in DEGREES
theta1 = 0.0 # coxa rotation
theta5 = 20.0 # femur
theta6 = 5.0 # tibia

data.qpos[:] = 0.0
data.qpos[0] = x_trunk0
data.qpos[1] = z_trunk0
data.qpos[2] = np.deg2rad(theta1)
data.qpos[3] = np.deg2rad(theta5)
data.qpos[4] = np.deg2rad(theta6)
data.qvel[:] = 0.0
mujoco.mj_forward(model, data)

```

Compress the spring by changing its rest length

```

In [25]: # --- length L_now at crouch as reference ---
L_now = float(data.ten_length[spring_id])

model.tendon_lengths[spring_id, 0] = L_now + delta
model.tendon_lengths[spring_id, 1] = L_now + delta

# velocities start at zero -> we stored potential energy only
data.qvel[:] = 0.0
mujoco.mj_forward(model, data)

```

Simulate

```

In [26]: duration = 2.0
fps = 120

t_hist = []
x_hist = []
z_hist = []

th1_hist = []
th5_hist = []
th6_hist = []

frames = []

with mujoco.Renderer(model, width=640, height=480) as rend:
    # preview initial configuration
    rend.update_scene(data)
    img0 = rend.render()

```

```

media.show_image(img0)

# Side-view camera
cam = mujoco.MjvCamera()
mujoco.mjv_defaultCamera(cam)
cam.azimuth = 90      # side view (+x to the right, z up)
cam.elevation = -15   # slight tilt down
cam.distance = 0.8     # far enough to see whole jump
cam.lookat[:] = np.array([0.25, 0.0, 0.05]) # center between the two legs

while data.time < duration:
    mujoco.mj_step(model, data)

    # COM position
    x_trunk = data.xipos[trunk_bid, 0]
    z_trunk = data.xipos[trunk_bid, 2]

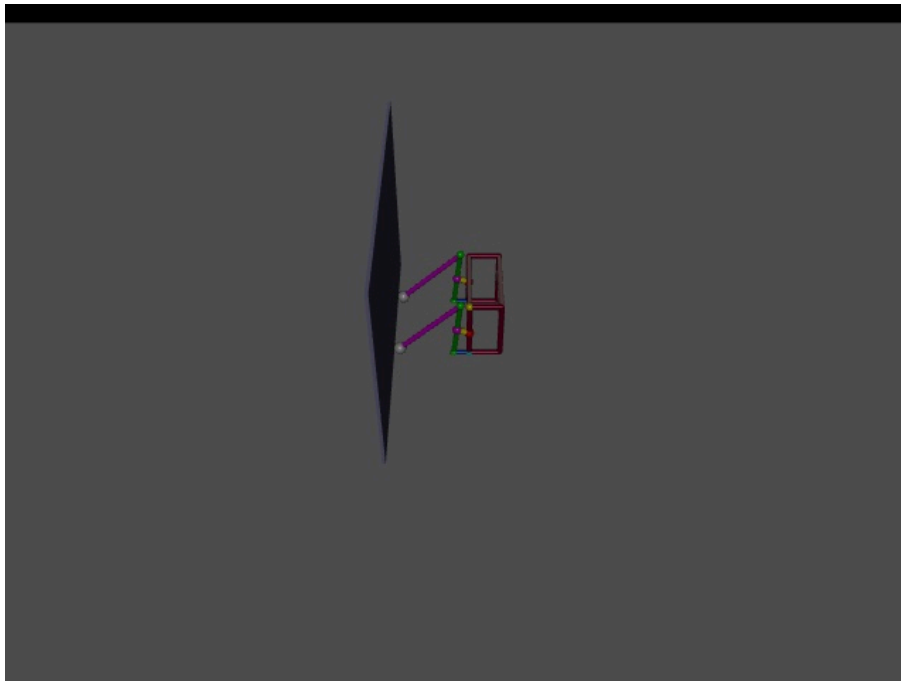
    t_hist.append(data.time)
    x_hist.append(x_trunk)
    z_hist.append(z_trunk)

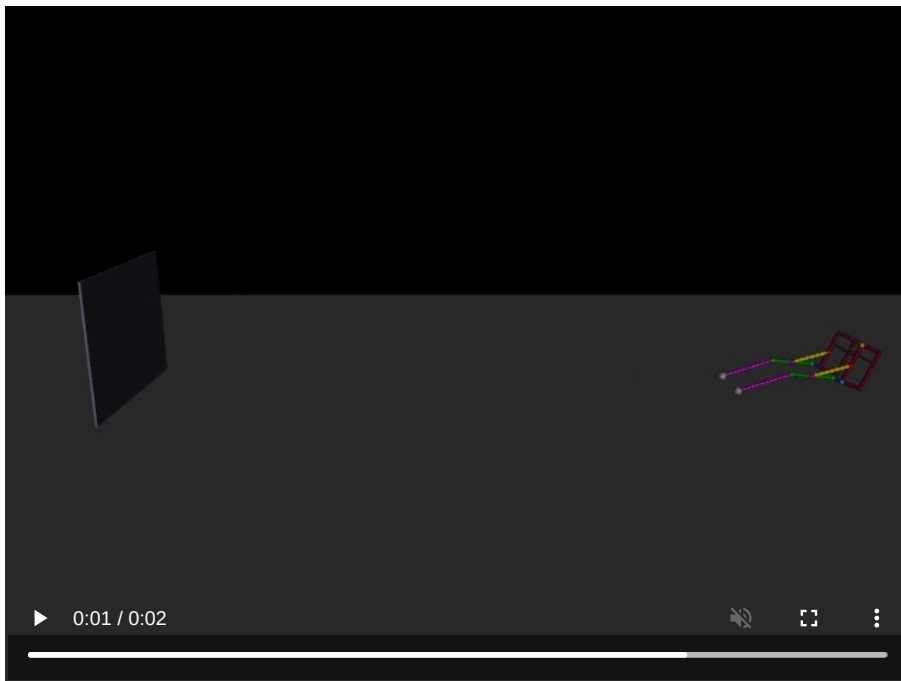
    th1_hist.append(np.rad2deg(data.qpos[2])) # theta1
    th5_hist.append(np.rad2deg(data.qpos[3])) # theta5
    th6_hist.append(np.rad2deg(data.qpos[4])) # theta6

    if len(frames) < data.time * fps:
        rend.update_scene(data, camera=cam)
        frames.append(rend.render())

media.show_video(frames, fps=fps)

```





Plots

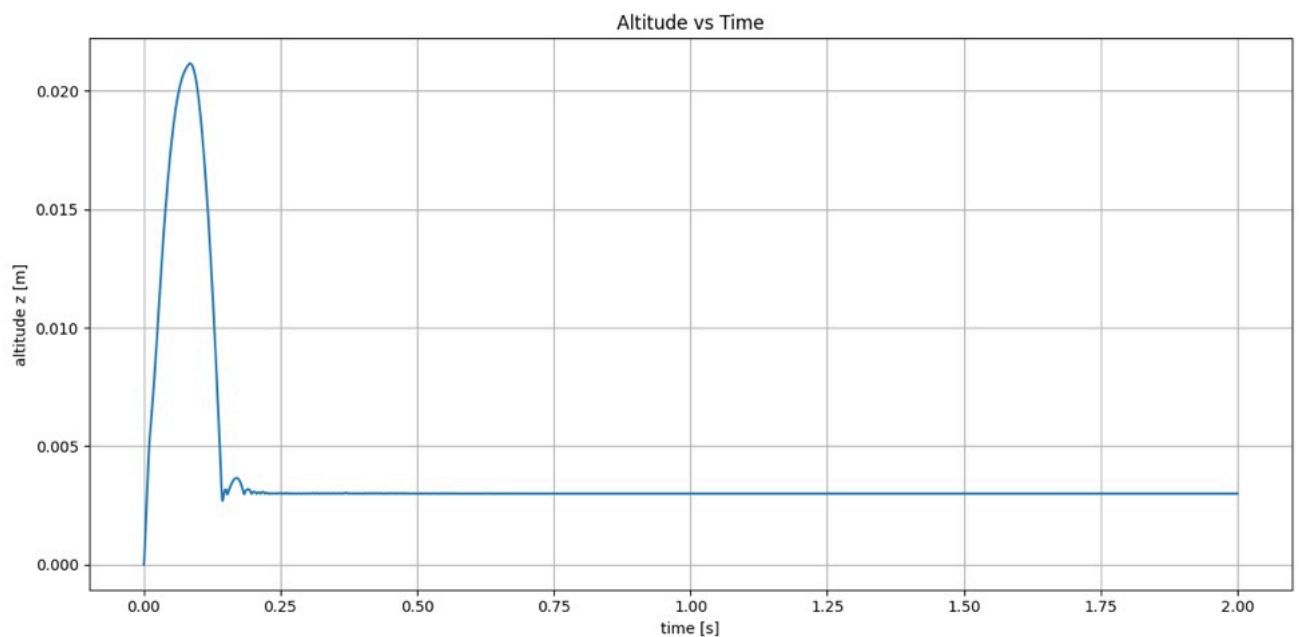
```
In [27]: t_hist = np.array(t_hist)
x_hist = np.array(x_hist)
z_hist = np.array(z_hist)

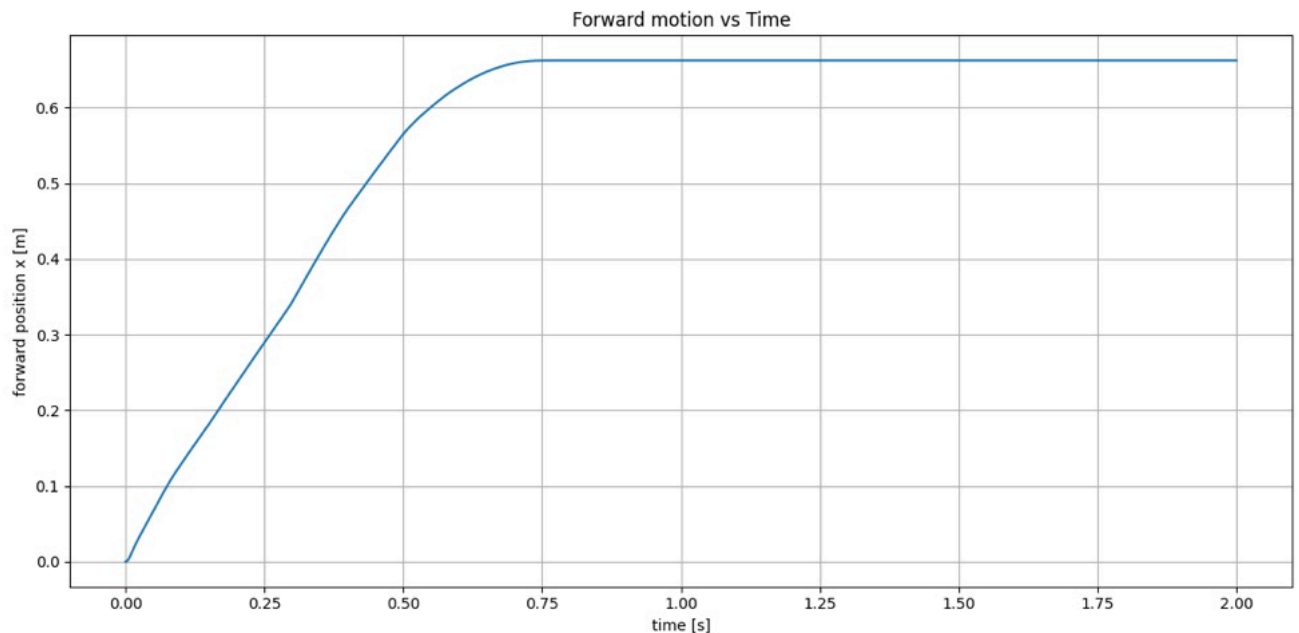
plt.figure(figsize=(12, 6))
plt.plot(t_hist, z_hist)
plt.title("Altitude vs Time")
plt.xlabel("time [s]")
plt.ylabel("altitude z [m]")
plt.grid(True)
plt.tight_layout()

plt.figure(figsize=(12, 6))
plt.plot(t_hist, x_hist)
plt.title("Forward motion vs Time")
plt.xlabel("time [s]")
plt.ylabel("forward position x [m]")
plt.grid(True)
plt.tight_layout()

plt.show()

print(f"Max altitude z: {z_hist.max():.4f} m")
print(f"Net forward displacement x: {x_hist[-1] - x_hist[0]:.4f} m")
```





Max altitude z: 0.0212 m
 Net forward displacement x: 0.6623 m

Optimization

We define a function that for a given spring position and stiffness simulate the jump

It is basically what we've done above encapsulated in a function.

```
In [28]: def simulate_jump(k_spring, alpha, beta, duration=2.0):
    xml = make_xml(alpha, beta, k_spring)
    model = mujoco.MjModel.from_xml_string(xml)
    data = mujoco.MjData(model)

    trunk_bid = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_BODY, "trunk")
    spring_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_TENDON, "Palpha_Pbeta_spring")

    data.qpos[:] = 0.0
    x_trunk0 = 0.0
    z_trunk0 = 0.0

    theta1 = 0.0
    theta5 = 20.0
    theta6 = 5.0

    data.qpos[0] = x_trunk0
    data.qpos[1] = z_trunk0
    data.qpos[2] = np.deg2rad(theta1)
    data.qpos[3] = np.deg2rad(theta5)
    data.qpos[4] = np.deg2rad(theta6)
    data.qvel[:] = 0.0
    mujoco.mj_forward(model, data)

    # ---- PRELOAD THE SPRING ----
    L_now = float(data.ten_length[spring_id])

    # stores energy 1/2 * k * (L_now - (L_now + delta_L))^2
    model.tendon_lengths[spring_id, 0] = L_now + delta
    model.tendon_lengths[spring_id, 1] = L_now + delta

    # re-forward to update forces with new rest length
    mujoco.mj_forward(model, data)

    max_z = -1e9
    max_x = -1e9

    while data.time < duration:
        mujoco.mj_step(model, data)
        x_trunk = data.xipos[trunk_bid, 0]
        z_trunk = data.xipos[trunk_bid, 2]
        if z_trunk > max_z:
            max_z = z_trunk
        if x_trunk > max_x:
            max_x = x_trunk

    return max_z, max_x
```

Spring placement optimization

We look for beta and alpha that maximize the forward motion distance

The stiffness is kept at $k = 150$

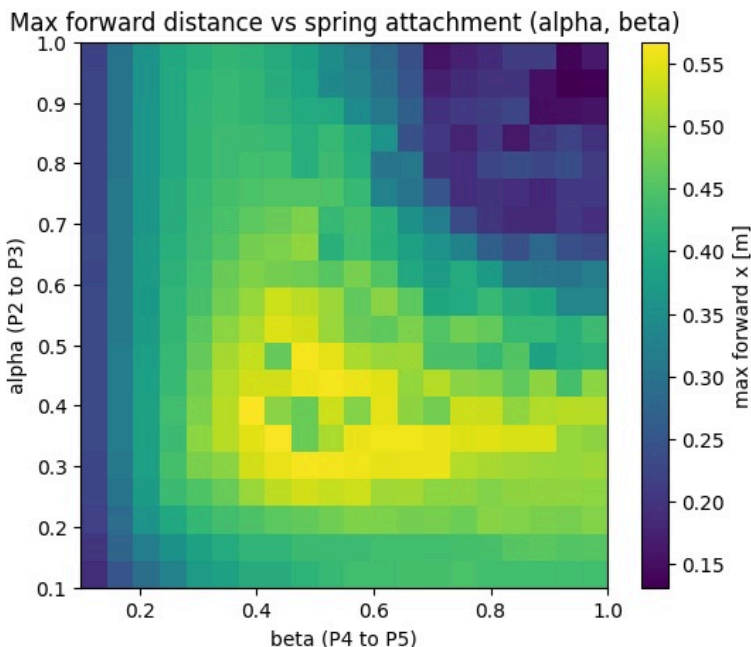
```
In [ ]: k_spring = 150
alphas = np.linspace(0.1, 1.0, 20)
betas = np.linspace(0.1, 1.0, 20)

max_z_grid = np.zeros((len(alphas), len(betas)))
max_x_grid = np.zeros((len(alphas), len(betas)))

for i, a in enumerate(alphas):
    for j, b in enumerate(betas):
        print(f"Sim ({i+1}/{len(alphas)}, {j+1}/{len(betas)}): alpha={a:.2f}, beta={b:.2f}")
        zmax, xmax = simulate_jump(k_spring, a, b, duration=0.5)
        max_z_grid[i, j] = zmax
        max_x_grid[i, j] = xmax
```

Forward distance heatmap

```
In [30]: plt.figure(figsize=(6,5))
plt.imshow(max_x_grid, origin="lower",
           extent=[betas[0], betas[-1], alphas[0], alphas[-1]],
           aspect="auto")
plt.colorbar(label="max forward x [m]")
plt.xlabel("beta (P4 to P5)")
plt.ylabel("alpha (P2 to P3)")
plt.title("Max forward distance vs spring attachment (alpha, beta)")
plt.show()
```



Retrieve α, β

```
In [31]: # Best for forward x
idx_flat_x = np.argmax(max_x_grid)
i_x, j_x = np.unravel_index(idx_flat_x, max_x_grid.shape)
alpha_star_x = alphas[i_x]
beta_star_x = betas[j_x]
print(f"Best forward x: alpha*={alpha_star_x:.2f}, beta*={beta_star_x:.2f}, x_max={max_x_grid[i_x,j_x]:.4f} m")
```

Best forward x: alpha*=0.38, beta*=0.38, x_max=0.5671 m

Refined the optimization

We look around the first optimals

```
In [ ]: alpha_m = max(alpha_star_x - 0.2, 0)
alpha_M = min(alpha_star_x + 0.2, 1)
beta_m = max(beta_star_x - 0.2, 0)
beta_M = min(beta_star_x + 0.2, 1)
alphas_opt = np.linspace(alpha_m, alpha_M, 20)
betas_opt = np.linspace(beta_m, beta_M, 20)

max_z_grid = np.zeros((len(alphas_opt), len(betas_opt)))
max_x_grid = np.zeros((len(alphas_opt), len(betas_opt)))

for i, a in enumerate(alphas_opt):
```

```

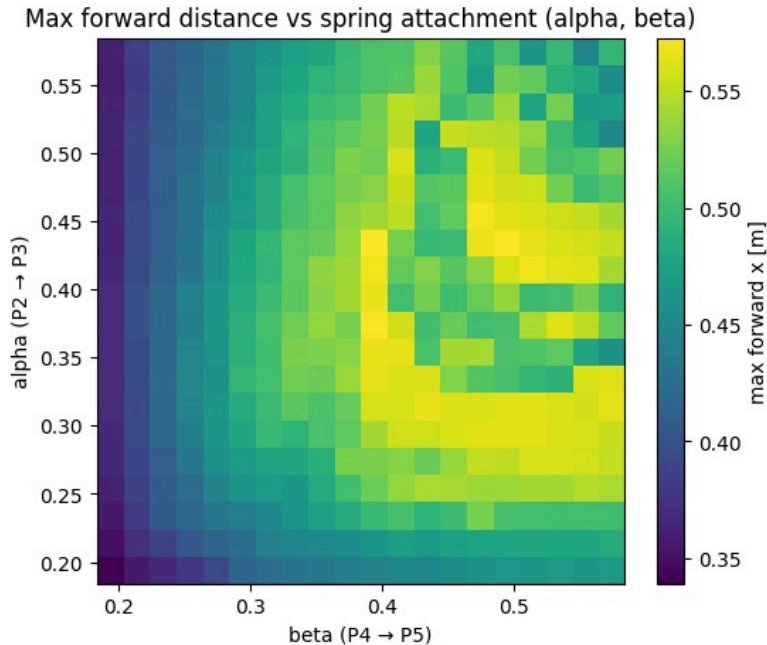
for j, b in enumerate(betas_opt):
    print(f"Sim ({i+1}/{len(alphas_opt)}, {j+1}/{len(betas_opt)}): alpha={a:.2f}, beta={b:.2f}")
    zmax, xmax = simulate_jump(k_spring, a, b, duration=0.5)
    max_z_grid[i, j] = zmax
    max_x_grid[i, j] = xmax

```

```

In [33]: plt.figure(figsize=(6,5))
plt.imshow(max_x_grid, origin="lower",
           extent=[betas_opt[0], betas_opt[-1], alphas_opt[0], alphas_opt[-1]],
           aspect="auto")
plt.colorbar(label="max forward x [m]")
plt.xlabel("beta (P4 → P5)")
plt.ylabel("alpha (P2 → P3)")
plt.title("Max forward distance vs spring attachment (alpha, beta)")
plt.show()

```



```

In [34]: idx_flat_x = np.argmax(max_x_grid)
i_x, j_x = np.unravel_index(idx_flat_x, max_x_grid.shape)
alpha_star_x = alphas_opt[i_x]
beta_star_x = betas_opt[j_x]
print(f"Best forward x: alpha*={alpha_star_x:.2f}, beta*={beta_star_x:.2f}, x_max={max_x_grid[i_x,j_x]:.4f} m")

```

Best forward x: alpha*=0.44, beta*=0.50, x_max=0.5729 m

Run a stiffness optimization

```

In [ ]: k_values = np.logspace(0, np.log10(150), 100)
z_max_list = np.zeros_like(k_values)
x_max_list = np.zeros_like(k_values)

for i, k in enumerate(k_values):
    print(f"{i+1}/{len(k_values)} k_spring = {k:.3e}")
    zmax, xmax = simulate_jump(k, alpha_star_x, beta_star_x, duration=0.5)
    z_max_list[i] = zmax
    x_max_list[i] = xmax

```

```

In [36]: # forward-distance-optimal k
idx_x = np.argmax(x_max_list)
k_star_x = k_values[idx_x]
x_star = x_max_list[idx_x]
print(f"Best forward x: k* = {k_star_x:.3e}, x_max = {x_star:.4f} m")

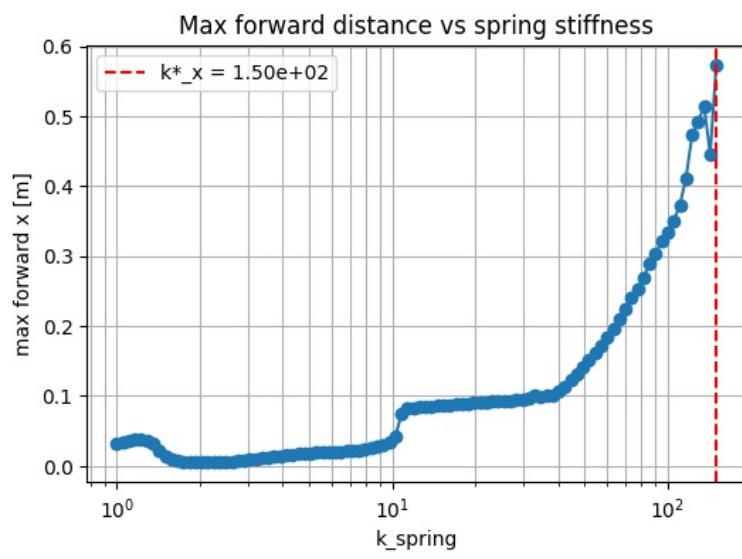
```

Best forward x: k* = 1.500e+02, x_max = 0.5729 m

```

In [37]: # --- Plots ---
plt.figure(figsize=(6,4))
plt.semilogx(k_values, x_max_list, "o-")
plt.axvline(k_star_x, color="r", linestyle="--", label=f"k*_x = {k_star_x:.2e}")
plt.xlabel("k_spring")
plt.ylabel("max forward x [m]")
plt.title("Max forward distance vs spring stiffness")
plt.grid(True, which="both")
plt.legend()
plt.show()

```



Future Experimental Design

To systematically validate the sensitivity of α and β in the physical domain, future work can use a continuous sliding rail mechanism rather than discrete attachment holes. This will allow for a gradient descent style tuning on the physical hardware.

This also opens the door to variable jump behavior with a single spring with fixed stiffness. Indeed, by tuning α and β at before the release, with data collection, we can achieve precise jump and landing.

Planning and Execution of the Manufacturing

Following the steps in chapter 51 of the reference book of the course (Manufacturing Workflow), a five layer fabrication method is chosen to fabricate the Hedgehog Flea-Inspired jumping robot's body and jumping leg mechanism. The following figure illustrates two versions of the CAD design for this robot in *LibreCAD* software. The flea's leg parts are mentioned in the illustrations for both designs. Note that the *Tarsus* leg is only considered in the second design to provide more friction for a better forward jump of the robot, while this leg was not considered in the first version (first prototype) of the robot, because it is mentioned in [20] that this leg does not provide any point to push during the flea's jump.

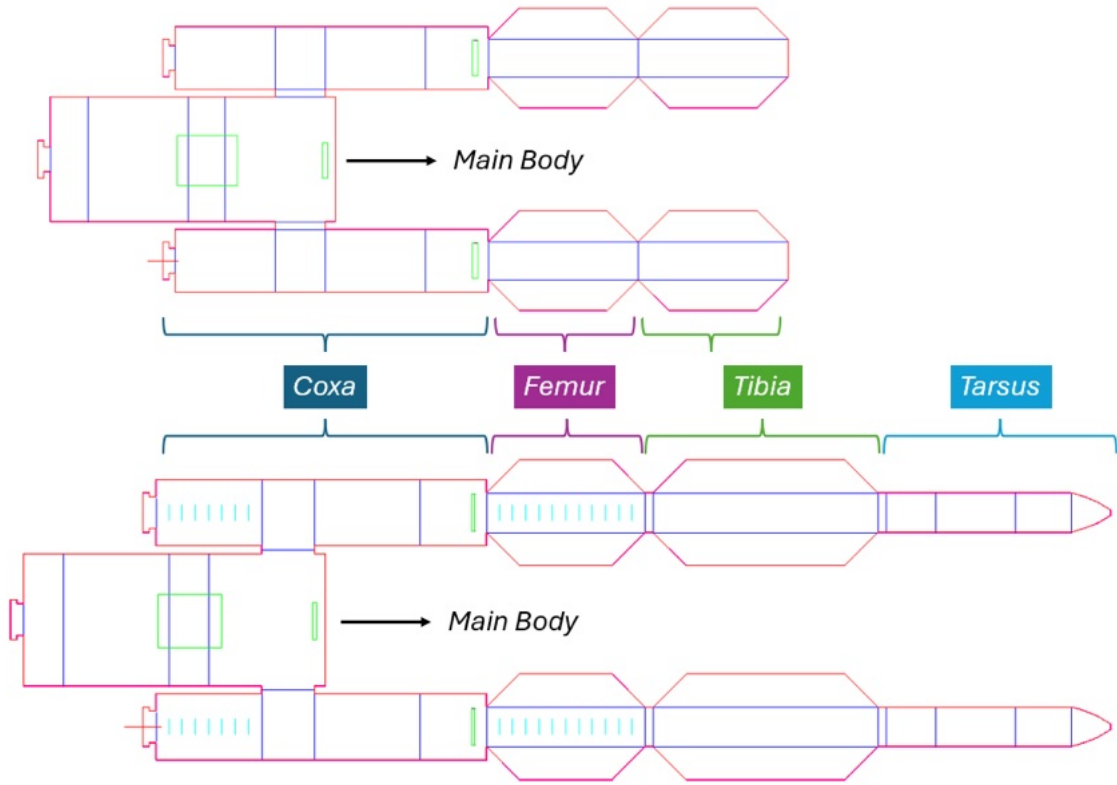


Figure 10: Two versions of the Hedgehog Flea-Inspired Jumping Robot's five-layer foldable design in LibreCAD - Different parts of the flea's leg are mentioned in both designs.

Looking at the figure above, it can be seen that the *Coxa* leg is designed as a four-bar mechanism which locks on one side using a T-lock, where the *Femur* and *Tibia* legs are designed as triangular shell members for increased moment of inertia providing more rigidity for the legs. The main body is also designed as a four-bar linkage that is being closed with a T-lock. However, since the motors are going to be installed inside the main body, it is not expected that this part demonstrates any bends or movements during the robot's jump. The figure below [11] shows the final cut of the five-layer fabrication of the flea-inspired jumping robot.

The folded body of the jumping robot is also demonstrated in Figure [12].

Pieces of bistable metal tape are chosen to act as spring at the joint between the *Coxa* and the *Femur* legs of the flea-inspired jumping mechanism. Although the bistable feature of these pieces of metal tapes are not used to help actuation in this project, but since cutting them in different sizes and shapes provides a range of stiffness values, we have used them as a way to modify the stiffness

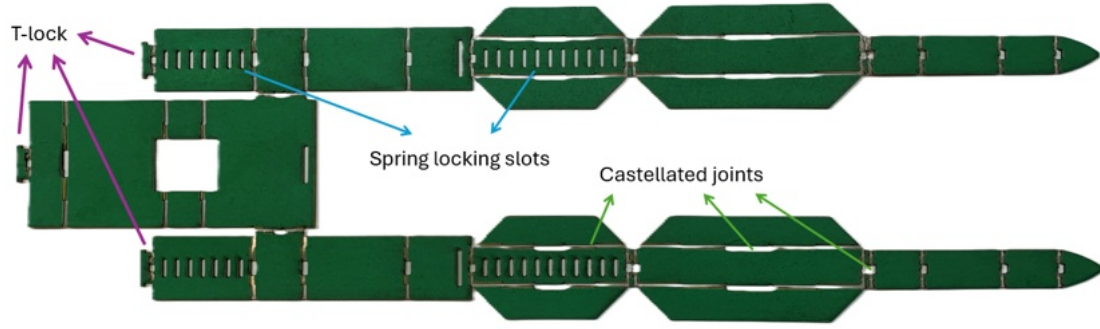


Figure 11: The final released five-layer foldable mechanism of the flea-inspired jumping robot after fabrication - The T-locks for the four-bar linkages and the slots designated for installing the springs between the two *Coxa* and *Femur* legs are demonstrated by arrows.

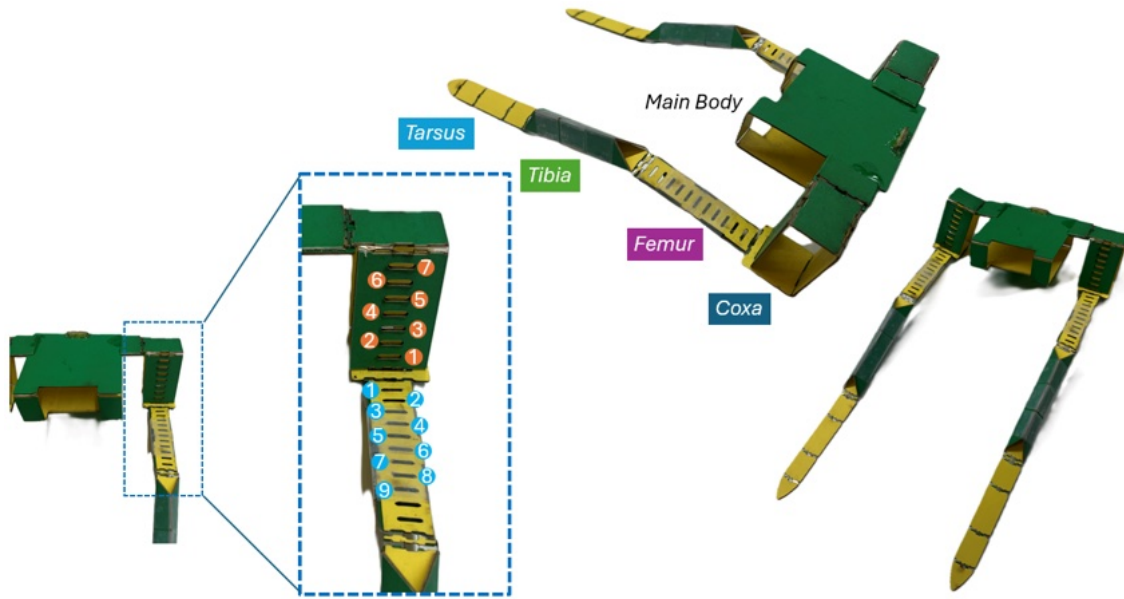


Figure 12: Folded body of the flea-inspired jumping robot - The numbered slots on the back of the *Coxa* (orange) and front of the *Femur* (light blue) legs are the fastening points for the springs between the legs.

of the joint for our study. In this study, three different lengths of 15mm-wide metal straps are used where the length values of 8 cm, 6 cm, and 4 cm correspond to the Soft, Medium, and Hard springs respectively. Figure 13 illustrates these three different sizes on the left, and the installation of the spring on the jumping leg on the right.

To contract the *Femur* leg towards *Coxa*, and not letting it to expand, a Ratchet Rack and Pinion mechanism is designed where the Ratchet Pinion is attached to the servo and pulls the Ratchet Rack to close the joint. But since the Ratchet Pinion has only teeth on 180 deg of its perimeter, after a half revolution, the Ratchet Rack is released and it can expand so the spring in between the joints reaches its free length (or angle here). A vertical slider guides the joint between the *Femur* and *Tibia* legs and translates the linear motion of the Rack to the joint's rotational + translational motion, and vice versa (depending on the source of actuation whether it comes from the servo or the spring). The designed Ratchet mechanism that is fabricated using 3D-printing and its use in the jumping robot leg is shown in Figure 14.

The final fabricated jumping robot is shown in Figure 15. It is good to mention here that the final version of the jumping robot with all its driving mechanisms weighs around 72 g which is much heavier than the bare body of weight 25 g.



Figure 13: Use of bistable metal tape as spring to provide variable joint stiffness between the *Coxa* and *Femur* legs of the flea-inspired jumping robot - The fastening points are shown inside red circles (right). Three different lengths of the bistable metal tape is used to provide three different stiffness values (left).

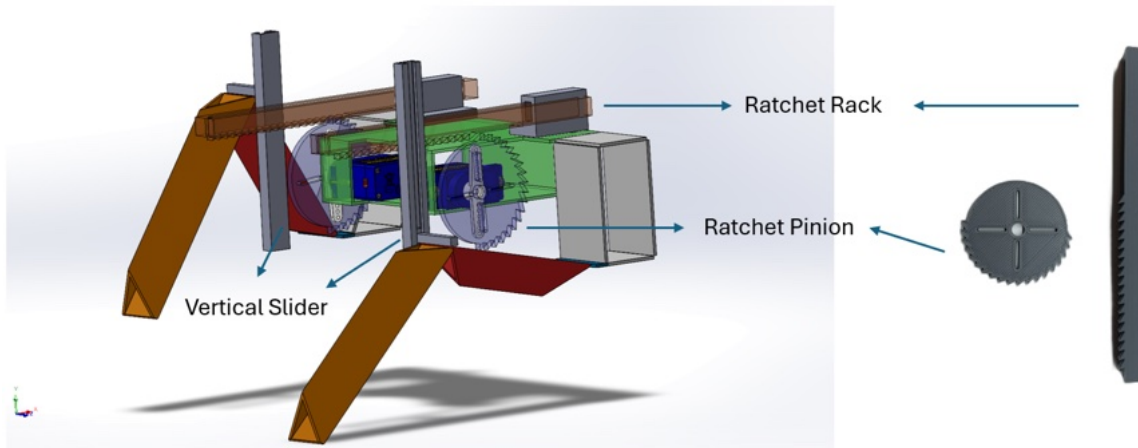


Figure 14: 180 deg Ratchet pinion and rack as the drive solution for contracting the jumping leg - The fabricated Ratchet pinion and rack using 3D printing (right), and the use of this mechanism in the designed model in *SolidWorks* (left)

Experimental Validation and Analysis

To perform jumping experiments on the fabricated robot, we tried to fix the Ratchet pinion and gear in a specific initial position so the servos can start pulling the *Femur* leg towards *Coxa* and contract the jumping muscle. However, considering the high minimum stiffness of the joint that makes the bare robot body to jump, the servos were not able to pull the rack back (even with the soft spring installed on the joint). Therefore, at this point, we decided to, at least, perform a passive jumping test on the fabricated body by manually contracting the joint and releasing it.

In these experiments, we used an *OptiTrack* motion capture system to measure the amount of the forward distance and the height in which the bare of the jumping robot travels during the jump with different springs installed on it. The weight of the robot in these experiments includes the weight of the fabricated five-layer body, the weight of the springs, and the weight of three motion tracking markers that are installed on the robot's main body. The weight of the trackers is equal to 19 g which is almost equal the weight of the two *Tower Pro SG90* servos. Figure 16 shows the robot body used in these passive jump experiments.

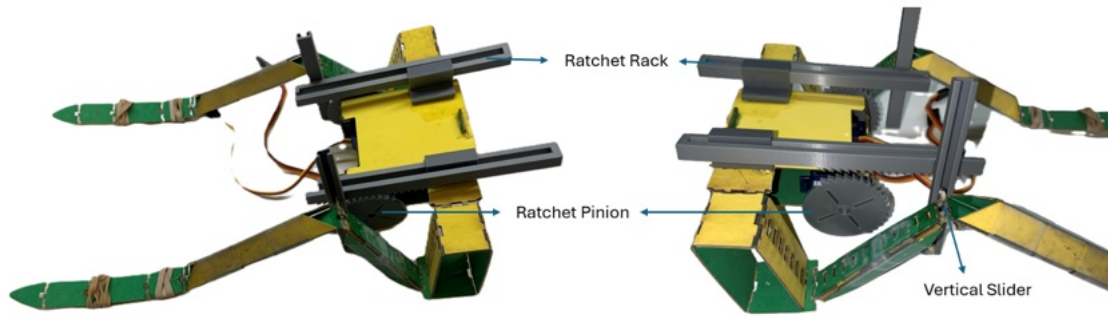


Figure 15: The final version of the fabricated jumping robot with full driving mechanism

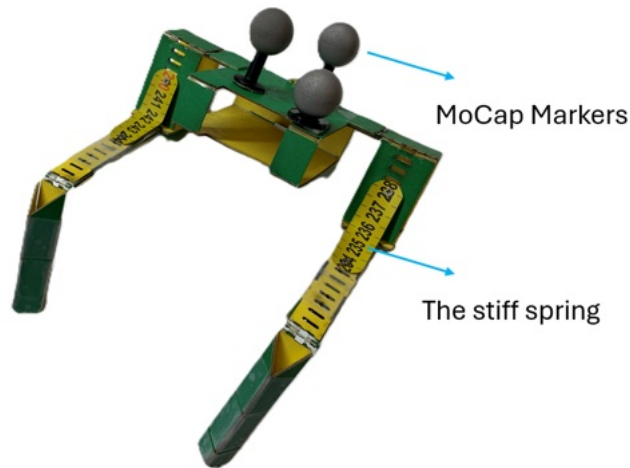


Figure 16: Flea-inspired jumping robot's body with motion capture markers mounted on it, used for the passive jumping test experiments

The experiments are done so that the back legs of the jumping mechanism (*Tibia*) are against an infinite mass (a thick wooden stick in this case), and the joint that drives the jump move is contracted by hand. When the hands are suddenly removed, the mechanism expands and tries to move in forward and up directions making the jumping motion.

The position data for the passive jumping experiments is collected using the motion capture system with a 120 Hz rate. The resulted data of each take is extracted separately in a .csv file and then the data inside it is cleaned (this was necessary due to some data loss occurred because of the hands blocking the MoCap markers) and plotted. The best take for each spring stiffness is then selected to be compare with others. The following plot depicts the resulted data for the forward x -axis and up z -axis motion of the jumping mechanism.

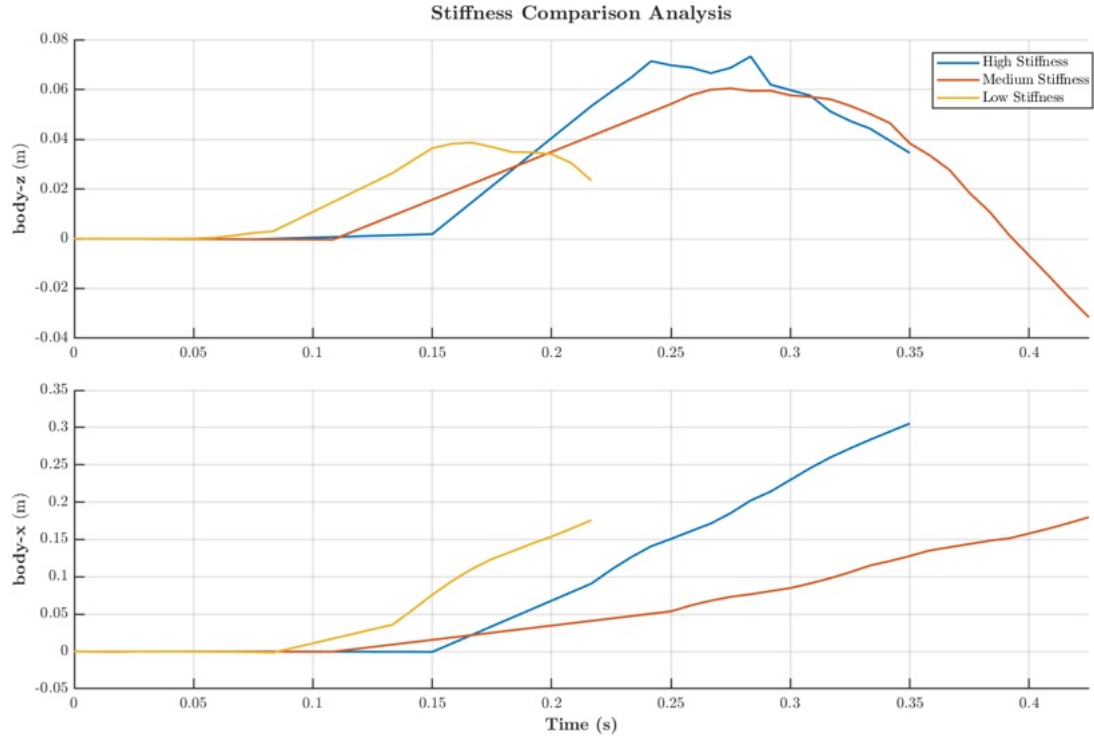


Figure 17: Results of the passive jump experiments for the flea-inspired jumping mechanism with *low*, *medium*, and *high* stiffness joint springs

From the plot above, it can be seen that the low-stiffness spring is not capable of pushing the robot body much in neither the forward, nor the up directions. However, both medium and high-stiffness springs are capable of throwing the robot's body in significant forward distances and make it have maximum relative heights as high as 8 *cm*. It can also be concluded from the diagrams that the speed in which the body starts to move forward, or jumps up is greater when we use a stiffer joint spring. However, to reach a better conclusion on the optimum stiffness for the jump and to obtain a reasonable compromise between the jumping length and height, a wider range of joint spring stiffness values must be tested.

Finally we provide a comparison plot between the low stiffness case and our MuJoCo simulation for behavior validation Figure [18](#).

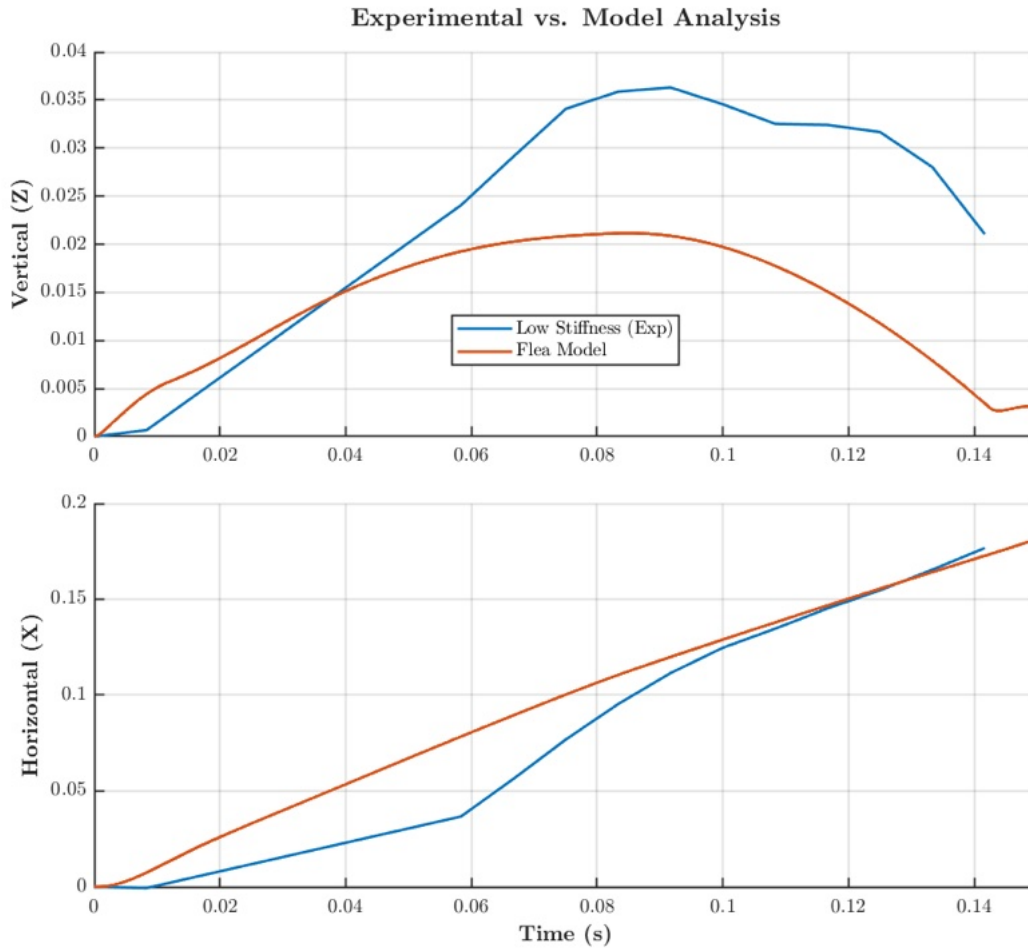


Figure 18: Vertical and Forward motion comparison between experiment and MuJoCo model.

Figure 18 shows that for the vertical motion, the timing of takeoff and landing match pretty accurately. A better result can be obtained by running further analysis and the mass and inertial influence on the jump height. For the forward motion, we can see that the linear trend is very accurate confirming that our MuJoCo model captures the forward dynamics accurately.

References

- [1] Longo, S. J., Cox, S. M., Azizi, E., Ilton, M., Olberding, J. P., and Patek, S. N., "Beyond power amplification: latch-mediated spring actuation is an emerging framework for the study of diverse elastic systems," *Journal of Experimental Biology*, vol. 222, no. 15, p. jeb197889, 2019.
- [2] Ilton, M., Bhamla, M. S., Ma, X., Cox, S. M., Fitchett, L., Kim, Y. S., Koh, J.-S., Krishnamurthy, D., Kuo, C.-W., Temel, F. Z. *et al.*, "The principles of cascading power limits in small, fast biological and engineered systems," *Science*, vol. 360, no. 6387, p. eaao1082, 2018.
- [3] Patek, S. N., "Latch-mediated spring actuation (lamsa): the power of integrated biomechanical systems," *Journal of Experimental Biology*, vol. 226, no. Suppl.1, p. jeb245262, 2023.
- [4] Sutton, G. P. and Burrows, M., "Biomechanics of jumping in the flea," *Journal of Experimental Biology*, vol. 214, no. 5, pp. 836–847, 2011.
- [5] Noh, M., Kim, S.-W., An, S., Koh, J.-S., and Cho, K.-J., "Flea-inspired catapult mechanism for miniature jumping robots," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1007–1018, 2012.
- [6] Koh, J. S., Jung, S. P., Noh, M., Kim, S. W., and Cho, K. J., "Flea inspired catapult mechanism with active energy storage and release for small scale jumping robot," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2013, pp. 26–31.

REFERENCES

- [7] Yun, R., Liu, Z., Leng, J., Huang, J., Cui, Y., Yan, X., and Qi, M., “A 3.4-millimeter flea-sized robot with powerful jumping and fast crawling locomotion,” *IEEE Robotics and Automation Letters*, vol. 8, no. 5, pp. 2868–2875, 2023.
- [8] Vincent, J. F. V. and Wegst, U. G. K., “Design and mechanical properties of insect cuticle,” *Arthropod Structure & Development*, vol. 33, no. 3, pp. 187–199, 2004.
- [9] Weis-Fogh, T., “Thermodynamic properties of resilin, a rubber-like protein,” *Journal of Molecular Biology*, vol. 3, pp. 520–531, 1961.
- [10] Andersen, S. L. and Weis-Fogh, T., “Resilin: A rubberlike protein in arthropod cuticle,” *Advances in Insect Physiology*, vol. 2, pp. 1–65, 1964.
- [11] Bolmin, O., Socha, J. J., Alleyne, M., Dunn, A. C., Fezzaa, K., and Wissa, A. A., “Nonlinear elasticity and damping govern ultrafast dynamics in click beetles,” *Proceedings of the National Academy of Sciences*, vol. 118, no. 5, p. e2014569118, 2021.
- [12] Carlson, J., Friedman, J., Kim, C., and Sung, C. R., “Rebound: Untethered origami jumping robot with controllable jump height,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2020, pp. 10 089–10 095.
- [13] Chen, F. and Aukes, D., “Direct encoding of tunable stiffness into an origami-inspired jumping robot leg,” *Journal of Mechanisms and Robotics*, vol. 16, no. 3, p. 031012, 2024.
- [14] Narang, Y. S., Vlassak, J. J., and Howe, R. D., “Mechanically versatile soft machines through laminar jamming,” *Advanced Functional Materials*, vol. 28, no. 17, p. 1707136, 2018.
- [15] Sadeghi, S., Allison, S., Betsill, B., and Li, S., “Exploiting the nonlinear stiffness of tmp origami folding to enhance robotic jumping performance,” 2020, arXiv preprint.
- [16] Zhai, Z., Wang, Y., Lin, K., Wu, L., and Jiang, H., “In situ stiffness manipulation using elegant curved origami,” *Science Advances*, vol. 6, no. 47, p. eabe2000, 2020.
- [17] Kim, Y.-J., Cheng, S., Kim, S., and Iagnemma, K., “A novel layer jamming mechanism with tunable stiffness capability for minimally invasive surgery,” *IEEE Transactions on Robotics*, vol. 29, no. 4, pp. 1031–1042, 2013.
- [18] Tang, L., Li, Y., and Li, B., “Moobot: A miniature origami omnidirectional jumping robot with high trajectory accuracy,” *IEEE Transactions on Industrial Electronics*, vol. 71, no. 6, 2024.
- [19] Yim, J. K., Singh, B. R. P., Wang, E. K., Featherstone, R., and Fearing, R. S., “Precision robotic leaping and landing using stance-phase balance,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3422–3429, 2020.
- [20] Sutton, G. P. and Burrows, M., “Biomechanics of jumping in the flea,” *Journal of Experimental Biology*, vol. 214, no. 5, pp. 836–847, 03 2011.
- [21] Todorov, E., Erez, T., and Tassa, Y., “Mujoco: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.